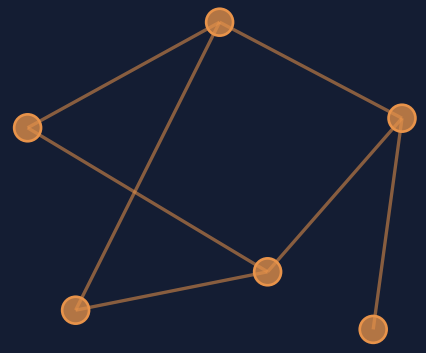
DİZİLER VE BELLEK TEMELLERİ

2. Hafta — Tek Boyutlu Diziler, Bellek Yerleşimi, İşaretçi ve Struct Temelleri

Karadeniz Teknik Üniversitesi

Yazılım Geliştirme Bölümü

Dr. Öğr. Üyesi Sayyad Alizadeh



Geçen Haftadan Bağlantı

- Veri yapısı ve ADT (soyut veri tipi) kavramlarını öğrendik.
- Big-O, Omega ve Theta ile bir algoritmanın büyüme hızını matematiksel olarak ifade etmeyi öğrendik.
- İkili arama üzerinde adım adım karmaşıklık analizi yaptık.
- **Bu hafta bu araçları ilk somut veri yapısına — diziye — uygulayacağız.**

Bu Hafta Neden Önemli?

Dizi; üzerinde adres hesaplama, karmaşıklık analizi ve algoritma tasarımını ilk kez somut bir veri yapısıyla uygulayacağımız yapıdır.

İşaretçi ve struct kavramları ise 3. haftada göreceğimiz Bağlı Listelerin doğrudan temelidir.

Bu Haftanın Öğrenme Hedefleri

Bu haftanın sonunda öğrenciler...

1. Dizilerin bellekte bitişik olarak nasıl yerleştiğini ve adres hesaplama formülünü uygulayabilir
2. C dilinde dizi, işaretçi ve struct tanımlayıp kullanabilir
3. Dizi üzerinde gezinme, ekleme, silme ve arama işlemlerini C ile gerçekleştirebilir
4. Ekleme/silme sırasında eleman kaydırmanın maliyetini adım adım izleyip karmaşıklığını hesaplayabilir
5. Öz-referanslı yapı kavramını açıklayıp bir sonraki haftaya (Bağlı Listeler) hazırlık yapabilir
6. Çok boyutlu dizi, seyrek matris, polinom ve union kavramlarını temel düzeyde tanıyabilir

Bu Haftanın Yol Haritası

ÇEKİRDEK

Tek boyutlu diziler, bellek/adres, işaretçi, struct, temel işlemler

Slayt 5–43

EK MATERYAL

Çok boyutlu diziler, seyrek matris, polinom, union (kısa)

Slayt 44–53

KAPANIŞ

Özet, gelecek hafta, quiz, ödev, kaynakça

Slayt 54–58

Ek Materyal bölümü kısa tutulmuştur; amacı farkındalık kazandırmaktır, ayrıntılı uygulama gerektirmez.

Dizi (Array) Nedir?

Aynı türden birden fazla değerin, bellekte ardışık (bitişik) olarak ve TEK bir isim altında saklanmasını sağlayan veri yapısıdır.



- 1. Haftada gördüğümüz ADT kavramının somut bir gerçekleştirimidir.
- İşlemler: oluştur, eriş, ekle, sil, gez, ara.
- **Her elemana İNDEKS (index) adı verilen bir tam sayı ile doğrudan erişilir.**

Neden Diziye İhtiyaç Duyarız?

Dizi Olmadan (5 ayrı değişken)

```
int not1 = 72;
int not2 = 85;
int not3 = 90;
int not4 = 61;
int not5 = 78;
```

- 5 not için 5 farklı isim gerekir.
- **100 öğrenci olsa 100 değişken mi tanımlayacağız?**

Dizi ile (tek yapı)

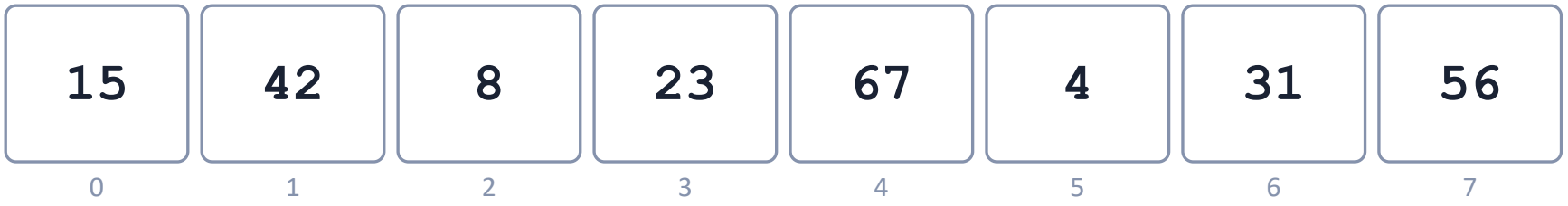
```
int notlar[5] = {72, 85, 90, 61, 78};

// Döngü ile tümüne erişim:
for (int i = 0; i < 5; i++)
    printf("%d\n", notlar[i]);
```

Tek isim (notlar) + indeks (i) ile sınırsız sayıda elemana düzenli erişilebilir.

Bellekte Bitişik (Ardışık) Yerleşim

- Dizinin tüm elemanları bellekte YAN YANA, aralarında boşluk olmadan yerleşir.



Bellek adresleri artan yönde, eşit aralıklarla ilerler →

Dizilerin bitişik bellek yerleşimi, herhangi bir elemana O(1) zamanda doğrudan erişim sağlar. İkili aramanın O(log n) zaman karmaşıklığı ise her adımda arama aralığının yarıya indirilmesinden kaynaklanır. Dizilerin doğrudan erişim özelliği bu işlemin verimli uygulanmasını sağlar.

Dizi Bildirimi — C Dilinde

Genel Sözdizimi

```
veri_turu dizi_adi[eleman_sayisi];
```

Örnek	Anlamı
`int notlar[5];`	5 elemanlık, başlangıçta belirsiz değerli tam sayı dizisi
`float fiyatlar[10];`	10 elemanlık ondalıklı sayı dizisi
`char isim[20];`	20 karakterlik metin (string) dizisi

Dikkat: Sabit boyutlu dizilerin kapasitesi oluşturulduktan sonra değiştirilemez. C dilinde değişken uzunluklu diziler (VLA) bazı sürümlerde desteklense de bunların boyutu da oluşturulduktan sonra değiştirilemez.

Dizi Başlatma (Initialization)

```
int notlar[5] = {72, 85, 90, 61, 78};
```

Tüm elemanlar açıkça belirtilir.

```
int notlar[5] = {72, 85};
```

Kalan elemanlar (index 2,3,4) otomatik olarak 0 olur.

```
int notlar[] = {72, 85, 90, 61, 78};
```

Boyut belirtilmez; derleyici eleman sayısından (5) çıkarır.

```
int notlar[5];
```

Başlatılmamış dizi — elemanlar ÇÖP (çarp değerler) içerir, kullanılmadan önce atama yapılmalıdır.

İndeks Kavramı

- **C'de dizi indeksleri 0'DAN başlar. n elemanlı bir dizinin geçerli indeksleri: 0, 1, ..., n−1.**



Yaygın Hata: 5 elemanlı bir dizide notlar[5]'e erişmeye çalışmak! Geçerli son indeks notlar[4]'tür. Bu, C'de derleyicinin YAKALAMADığı (tanımsız davranış üreten) ciddi bir hatadır.

Bu dersteki tüm C kodlarında dizi sınırları dikkatle kontrol edilecek ve geçersiz indeks kullanılmayacaktır.

Adres Hesaplama Formülü

- Bitişik yerleşim sayesinde herhangi bir elemanın adresi, DÖNGÜ GEREKMEDEN doğrudan hesaplanabilir.

```
adres(i) = taban_adres + i × eleman_boyutu
```

`taban_adres`

Dizinin ilk elemanının (indeks 0) bellek adresi

`i`

Erişilmek istenen elemanın indeksi

`eleman_boyutu`

Her elemana ayrılan bayt sayısı (ör. int için 4 bayt)

Bu formül tüm bu dersteki dizi konularının temelidir — ezberlemeniz önerilir.

Adres Hesaplama — Örnek (1/2)

int dizi[8] = {15,42,8,23,67,4,31,56}; taban_adres=1000 sizeof(int)=4

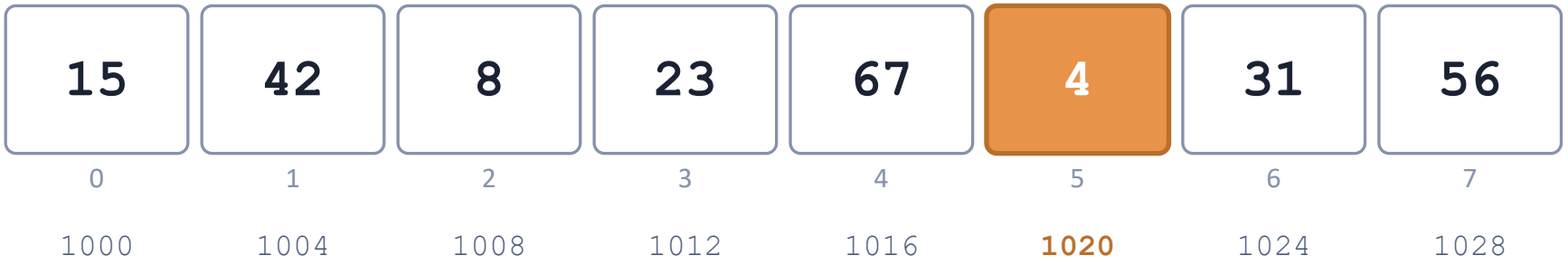


indeks (üstte) / adres (altta, bayt)

i	Değer	Hesap	Adres
0	15	1000+0×4	1000
1	42	1000+1×4	1004
2	8	1000+2×4	1008
3	23	1000+3×4	1012
4	67	1000+4×4	1016
5	4	1000+5×4	1020
6	31	1000+6×4	1024
7	56	1000+7×4	1028

Adres Hesaplama — Örnek (2/2)

Soru: dizi[5] elemanının adresi nedir?




```
adres(5) = taban_adres + i × eleman_boyutu
```

```
adres(5) = 1000 + 5 × 4
```

```
adres(5) = 1000 + 20
```

adres(5) = 1020 *— slayt 12'deki tablo ile doğru orantılı*

sizeof Operatörü ve Eleman Boyutu

- eleman_boyutu değeri, C'nin sizeof operatörü ile hesaplanır — elle tahmin edilmemelidir.

Veri Türü	sizeof(...)	Tipik Boyut
`char`	`sizeof(char)`	1 bayt
`int`	`sizeof(int)`	4 bayt
`float`	`sizeof(float)`	4 bayt
`double`	`sizeof(double)`	8 bayt

Önemli: Boyutlar derleyiciye ve platforma göre değişebilir. Kodunuzda her zaman sizeof(dizi_adi[0]) ya da sizeof(veri_turu) kullanın — sabit sayı (4, 8 gibi) yazmayın.

Dizi ile İşaretçi İlişkisi

Bir sonraki bölüme köprü

- **C'de bir dizinin adı, çoğu ifadede ilk elemanının adresine eşdeğer sayılır.**

```
int dizi[5] = {15, 42, 8, 23, 67};

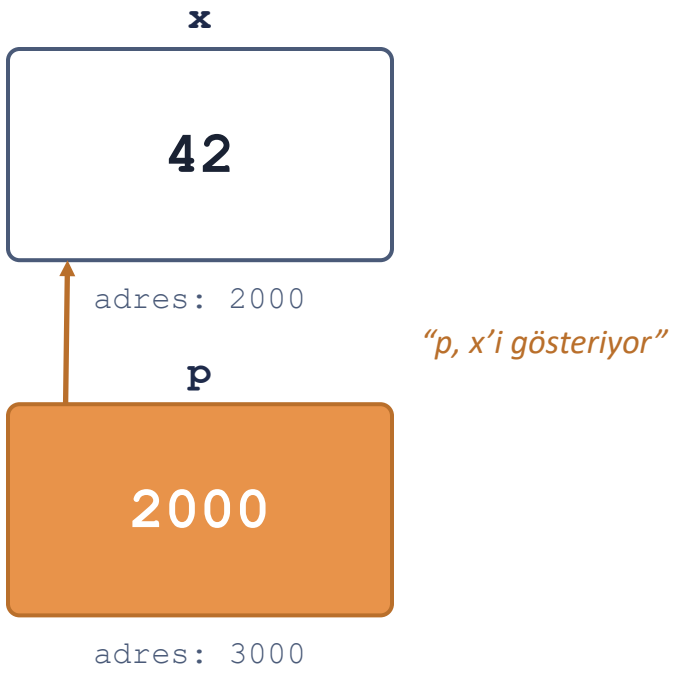
// Aşağıdaki iki ifade AYNI adresi verir:
printf("%p", (void*)dizi);      // dizi adı
printf("%p", (void*)&dizi[0]);  // ilk elemanın adresi
```

Bu eşdeğerlik, dizilerin işaretçilerle nasıl bağlantılı olduğunun temelidir. Şimdi işaretçi kavramını sıfırdan ele alacağız.

İşaretçi (Pointer) Nedir?

İşaretçi; bir değer değil, BAŞKA BiR DEĞİŞKENİN BELLEK ADRESİNİ tutan özel bir değişkendir.

- x normal bir değişken: değeri 42, kendi adresi 2000.
- p bir işaretçi: değeri 2000 (yani x'in adresi), kendi adresi ise 3000.
- **“p, x’i gösteriyor” demek, p’nin içinde x’in adresinin saklı olduğu anlamına gelir.**



Pointer Bildirimi ve Kullanımı — C'de

```
int x = 42;
int *p;       // p, int'e işaretçi
p = &x;       // p, x'in adresini alır

printf("%d", x);   // 42
printf("%d", *p);  // 42 (p'nin değeri)
printf("%p", (void*)p); // Örnek adres: 2000
```

Sembol	Anlamı
`&x`	x değişkeninin ADRESİNİ al (address-of)
`*p`	p'nin GÖSTERDİĞİ değeri al (dereference)
`int *p`	p, bir int'e işaretçi olarak bildirilir

Not: Bellek adresleri örnek olarak verilmiştir. Programın gerçek çıktısındaki adresler çalışma ortamına göre değişebilir.

Yaygın Hata: Başlatılmamış (NULL olmayan, çöp adresli) bir işaretçiyi *p ile kullanmak programın çökmesine (segmentation fault) yol açabilir.

Pointer Aritmetiği ve Diziler

- Bir işaretçiye 1 eklemek, bellekte 1 BAYT değil, 1 ELEMAN kadar ilerletir (derleyici veri türüne göre otomatik hesaplar).




```
int dizi[4] = {15, 42, 8, 23};
int *p = dizi;              // p = &dizi[0] = 3000
printf("%d", *(p + 2));     // dizi[2]'ye eşdeğer -> 8
```

Struct (Yapı) Nedir?

Struct; farklı TÜRLERDEN verileri TEK bir çatı altında toplayan, kullanıcı tanımlı bir veri türüdür.

```
numara: 123

isim: "Ayşe"

not: 87.5
```

- Örnek: bir öğrenci kaydı numara(int), isim(metin) ve not(ondalık) gibi farklı türden alanlar içerir.
- **Dizi tüm elemanları AYNI türden tutar; struct ise FARKLI türleri bir arada tutar — temel fark budur.**

Struct Tanımlama ve Kullanımı

```
struct Ogrenci {
    int numara;
    char isim[20];
    float not;
};

struct Ogrenci ogr1;
ogr1.numara = 123;
ogr1.not = 87.5;
```

Bellek Yerleşimi (ogr1)



Toplam boyut: 28 bayt (4 + 20 + 4)

Her alan (field), struct içinde kendi türüne göre ardışık bir offset (başlangıç konumu) alır — tıpkı dizideki adres hesabına benzer bir mantık.

Alanlara erişim: değişken_adı.alan_adı (ör. ogr1.numara). İşaretçi üzerinden erişimde ise -> operatörü kullanılır (bir sonraki slaytta görülecek).

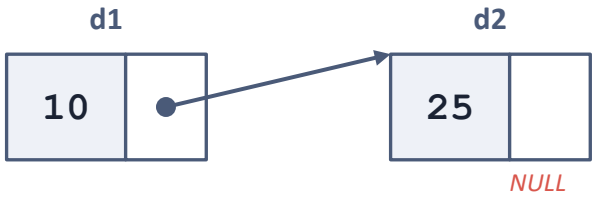
Öz-Referanslı Yapılar

3. Haftaya (Bağlı Listeler) köprü

```
struct Dugum {
    int veri;
    struct Dugum *sonraki;
};

struct Dugum d1, d2;
d1.veri = 10;
d1.sonraki = &d2;       // d1, d2'yi gösteriyor

struct Dugum *p = &d1;
printf("%d", p->veri); // -> ile erişim -> 10
```



Bir struct, KENDİ TÜRÜNDEN bir işaretçi alanı içerebilir. Bu, bir yapının bir sonraki aynı türden yapıyı göstermesini sağlar — zincirleme yapıların (Bağlı Liste) temelidir.

Bu görsel kural (veri | işaretçi kutusu) 3. haftada Bağlı Listeler konusunda aynen kullanılmaya devam edecektir.

Dizide Gezinme (Traversal)

Pseudocode

```
Fonksiyon Gezin(dizi, n)
    i ← 0
    iken i < n
        yazdır dizi[i]
        i ← i + 1
```

Her eleman TAM OLARAK bir kez ziyaret edilir → n eleman için n adım gerekir.

Karmaşıklık: **T(n) = n → O(n)** *(her durumda — en iyi, en kötü, ortalama — aynıdır)*

Dizide Gezinme — C Kodu ve Çıktı

```c
void gezin(int dizi[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", dizi[i]);
    }
    printf("\n");
}
```

Örnek Çalışma

```c
int notlar[5] = {72, 85, 90, 61, 78};
gezin(notlar, 5);

// Çıktı:
// 72 85 90 61 78
```

Dikkat: fonksiyona diziyi geçirirken boyutu (n) da AYRICA parametre olarak vermek gerekir — C, dizinin kaç eleman içerdiğini kendisi bilemez (sizeof, fonksiyon içinde işe yaramaz).

Dizide Arama — Geçen Haftadan Hatırlatma

- Geçen hafta ikili arama üzerinde derinlemesine işlediğimiz arama, tam olarak diziler üzerinde çalışır.
- **Dizi SİRALI değilse → doğrusal arama, O(n)**
- **Dizi SİRALIysa → ikili arama, O(log n)**

Bu haftaki ekleme/silme işlemlerinin dizinin SİRASINI bozabileceğini, bu yüzden sıralılık gerektiren ikili arama kullanılacaksa dizinin sıralı tutulması gerektiğini aklınızda tutun.

Detaylı pseudocode, C kodu ve adım adım örnekler için 1. Hafta sunumuna bakınız.

Dizi Sonuna Eleman Ekleme

Pseudocode

```
Fonksiyon SonaEkle(dizi, n, kapasite, deger)
    eğer n == kapasite
        dön HATA  // dizi dolu
    dizi[n] ← deger
    n ← n + 1
    dön n
```

C Kodu

```c
int sonaEkle(int dizi[], int n,
             int kapasite, int deger) {
    if (n == kapasite) {
        return -1;  // dizi dolu
    }
    dizi[n] = deger;
    return n + 1;
}
```

En basit ekleme durumu: O(1) *— kaydırma gerekmez, doğrudan boş hücreye yazılır. Kapasite kontrolü şarttır!*

Diziye Ortaya Eleman Ekleme

Problem Tanımı



- Kapasite = 8, mevcut eleman sayısı n = 6 olan bir dizimiz var.
- **Hedef: değer=99'u KONUM 2'ye ekleyip, o konumdan sonraki tüm elemanları bir sağa kaydırmak.**
- Bu, dizinin en karmaşık temel işlemidir — sonraki 4 slaytta adım adım izleyeceğiz.

Ortaya Ekleme — Pseudocode

```
Fonksiyon OrtayaEkle(dizi, n, kapasite, konum, deger)
    eğer n == kapasite
        dön HATA   // dizi dolu
    i ← n - 1
    iken i ≥ konum
        dizi[i+1] ← dizi[i]
        i ← i - 1
    dizi[konum] ← deger
    n ← n + 1
    dön n
```

Ortaya Ekleme — C Kodu

```
int ortayaEkle(int dizi[], int n, int kapasite,
               int konum, int deger) {
    if (n == kapasite) {
        return -1;              // dizi dolu
    }
    for (int i = n - 1; i >= konum; i--) {
        dizi[i + 1] = dizi[i]; // sağa kaydır
    }
    dizi[konum] = deger;
    return n + 1;
}
```

Ortaya Ekleme — Adım 1 / 4

Başlangıç Durumu



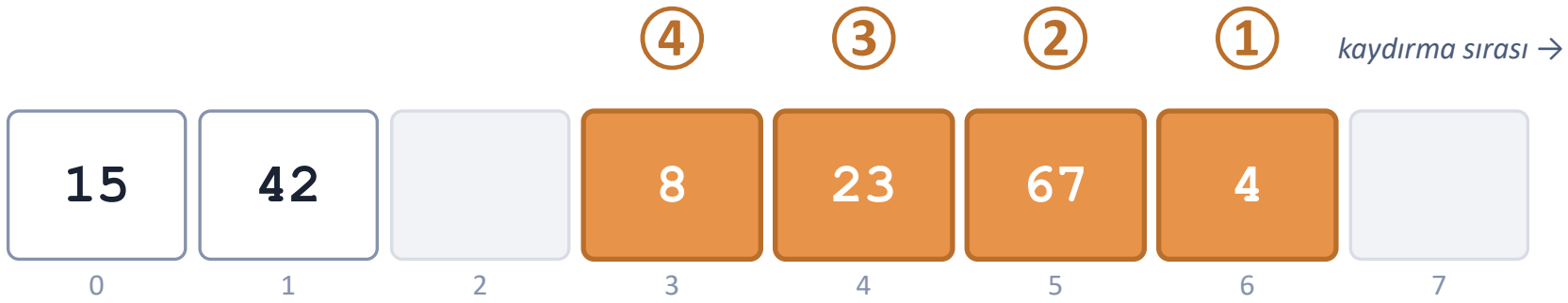
Dizide n=6 eleman var, kapasite 8. Konum 2'deki mevcut değer (8) ve sonrasındaki tüm elemanlar bir sağa kaymalı.

Kodda karşılık gelen satır

```
for (int i = n - 1; i >= konum; i--) {
    dizi[i + 1] = dizi[i];
}
```

Ortaya Ekleme — Adım 2 / 4

Elemanlar Sağa Kaydırılıyor (sondan başa doğru)



ÖNEMLİ: Kaydırma SONDAN BAŞA doğru yapılır (① index5→6, ② 4→5, ③ 3→4, ④ 2→3). Ters sırada yapılsaydı, veriler üst üste yazılıp KAYBOLURDU!

Kodda karşılık gelen satır

```
for (int i = n - 1; i >= konum; i--) {
    dizi[i + 1] = dizi[i];  // her yinelemede bir kaydırma
}
```

Ortaya Ekleme — Adım 3 / 4

Boşalan Hücre



konum 2 artık boş ve hazır

4 kaydırma işlemi tamamlandı. İndeks 2'nin eski değeri (8) artık indeks 3'te güvenle duruyor. İndeks 2 yeni değer için tamamen boş.

Kodda karşılık gelen satır

```
// döngü bitti (i, konum'un altına indi)
dizi[konum] = deger;   // bir sonraki adımda çalışır
```

Ortaya Ekleme — Adım 4 / 4

Yeni Eleman Yerleştirildi — Son Durum



✓ dizi[2] = 99 yerleşti. n = 7 oldu (6→7). İndeks 7 hala kullanılmayan kapasite.

Kodda karşılık gelen satır

```
dizi[konum] = deger;    // dizi[2] = 99
return n + 1;           // 6 + 1 = 7
```

Ortaya Ekleme — Karmaşıklık Analizi

En İyi Durum	En Kötü Durum	Ortalama Durum
Sona ekleme (konum = n) → 0 kaydırma	Başa ekleme (konum = 0) → n kaydırma	Konum eşit olasılıkla değişirse → n/2 kaydırma
`O(1)`	`O(n)`	`O(n)`

Saydığımız işlem: her bir dizi[i+1]=dizi[i] kaydırması. Karşılaştırma yok, sadece atama sayılıyor.

Dizilerde başa veya ortaya eleman eklemek, elemanların kaydırılmasını gerektirdiğinden O(n) zaman alabilir. Bağlı listelerde ise ekleme konumu önceden biliniyorsa bağlantılar O(1) zamanda güncellenebilir. Ancak ekleme konumunu bulmak gerekiyorsa toplam işlem O(n) zaman alabilir.

Diziden Eleman Silme

Problem Tanımı — az önce eklediğimiz dizi ile devam ediyoruz



- Şu anki dizimiz: n=7 [15,42,99,8,23,67,4]. Konum 1'deki değerin (42) silinmesi isteniyor.
- **Silme sonrası boşluk kalmaması için, konum 1'den sonraki tüm elemanlar bir SOLA kaydırılmalıdır.**

Silme — Pseudocode

```
Fonksiyon Sil(dizi, n, konum)
    eğer konum < 0 veya konum ≥ n
        dön HATA   // geçersiz konum
    i ← konum
    iken i < n - 1
        dizi[i] ← dizi[i+1]
        i ← i + 1
    n ← n - 1
    dön n
```

Silme — C Kodu

```c
int sil(int dizi[], int n, int konum) {
    if (konum < 0 || konum >= n) {
        return -1;              // geçersiz konum
    }
    for (int i = konum; i < n - 1; i++) {
        dizi[i] = dizi[i + 1];  // sola kaydır
    }
    return n - 1;
}
```

Silme — Adım 1 / 3

Başlangıç Durumu



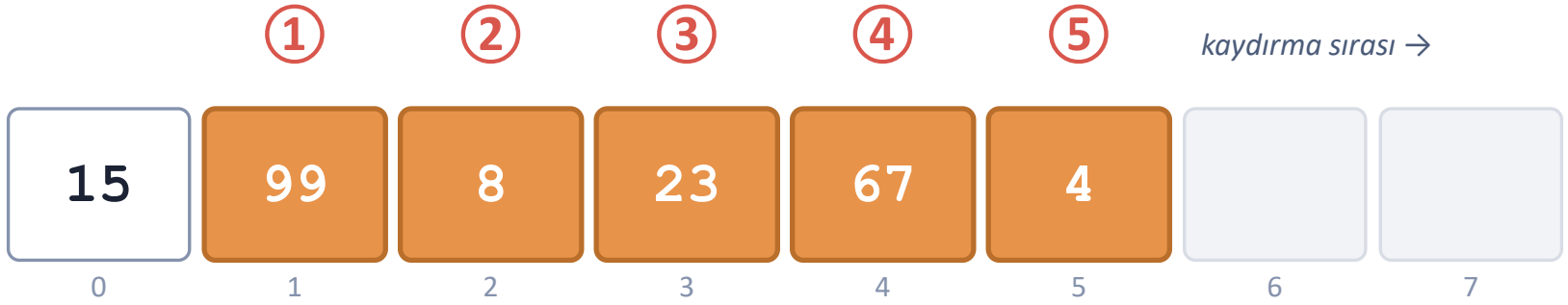
n=7. Konum 1'deki değer (42) kaldırılacak; konum 2'den (99) itibaren tüm elemanlar bir SOLA kayacak.

Kodda karşılık gelen satır

```
for (int i = konum; i < n - 1; i++) {
    dizi[i] = dizi[i + 1];
}
```

Silme — Adım 2 / 3

Elemanlar Sola Kaydırılıyor (baştan sona doğru)



Bu kez kaydırma BAŞTAN SONA yapılır (① idx2→1, ② 3→2, ③ 4→3, ④ 5→4, ⑤ 6→5) — eklemenin tam TERSİ! Çünkü artık SAĞDAN veri okuyup SOLA yazıyoruz, üzerine yazma riski yok.

Kodda karşılık gelen satır

```
for (int i = konum; i < n - 1; i++) {
    dizi[i] = dizi[i + 1];  // her yinelemede bir kaydırma
}
```

Silme — Adım 3 / 3

Son Durum



✓ 42 silindi. Dizi artık [15,99,8,23,67,4]. n=6 oldu (7→6). İndeks 6 ve 7 artık kullanılmayan kapasite.

Kodda karşılık gelen satır

```
// döngü bitti (i, n-1'e ulaştı)
return n - 1;   // 7 - 1 = 6
```

Silme — Karmaşıklık Analizi

En İyi Durum	En Kötü Durum	Ortalama Durum
Son elemanı silme (konum = n−1) → 0 kaydırma	İlk elemanı silme (konum = 0) → n−1 kaydırma	Konum eşit olasılıkla değişirse → (n−1)/2 kaydırma
`O(1)`	`O(n)`	`O(n)`

Ekleme ve silme, aynı karmaşıklık profilini paylaşır: başa yakın işlemler pahalı, sona yakın işlemler ucuzdur. Bu simetri, dizinin “ortadan değiştirme maliyeti yüksek” doğasının doğrudan sonucudur.

Dizinin Avantajları

Hızlı İndeksli Erişim

dizi[i] ile O(1) sürede doğrudan erişim — adres formülü sayesinde.

Bellek Verimliliği

Ekstra işaretçi alanı gerekmez; sadece veriler saklanır.

Basit ve Öngörülebilir

Sözdizimi ve davranışı anlaşılması kolaydır.

İyi Önbellek Performansı

Bitişik yerleşim, işlemcinin önbellek (cache) mekanizmasından verimli faydalanmasını sağlar.

Dizinin Sınırlamaları

Sabit Boyut

Kapasite derleme anında belirlenir; çalışma zamanında büyütülemez.

Pahalı Ekleme/Silme

Ortadaki bir işlem O(n) kaydırma maliyeti getirir (bu hafta gördük).

Bellek İsrafı Riski

Kapasite fazla ayrılırsa kullanılmayan boş hücreler bellek harcar.

Bu sınırlamaların hepsi TEK bir kök nedene dayanır: dizinin bitişik yerleşimi. Bir sonraki hafta göreceğimiz Bağlı Liste, bitişikliği terk ederek bu sorunların çoğunu çözer — ama kendi yeni değiş-tokuşlarıyla birlikte.

Önizleme: Dizi vs Bağlı Liste

3. Hafta için bir bakış

Özellik	Dizi	Bağlı Liste (3. Hafta)
İndeksli erişim	O(1)	O(n)
Başa/ortaya ekleme	O(n)	O(1) / O(n)*
Boyut esnekliği	Sabit	Dinamik
Bellek fazlası	Yok	Her düğümde işaretçi alanı

**Bağlı listede başa ekleme O(1)'dir; belirli bir konuma ekleme ise o konumu bulmak için yine O(n) gerektirir. Detaylar 3. hafta.*

İleri Düzey Ek Materyal

Bu bölüm KİSA tutulmuştur

- Çok Boyutlu Diziler
- Seyrek Matrisler (Sparse Matrix)
- Polinom Gösterimi
- Union Kavramı

Amaç farkındalık kazandırmaktır; ayrıntılı uygulama bu haftanın kapsamı dışındadır.

Çok Boyutlu Diziler — Kavram

- 2 boyutlu dizi, satır ve sütunlardan oluşan bir tablo gibi düşünülebilir: int m[3][4];

m[0][0]	m[0][1]	m[0][2]	m[0][3]
m[1][0]	m[1][1]	m[1][2]	m[1][3]
m[2][0]	m[2][1]	m[2][2]	m[2][3]

Kavramsal Görünüş (satır × sütun)

Bellekte Gerçek Yerleşim (Satır-Öncelikli)



Diziler bellekte HER ZAMAN tek boyutlu gibi, satır satır ardışık saklanır.

Önemli: m[1][2] gibi bir erişim, arka planda TEK bir doğrusal bellek şeridi üzerinde hesaplanır. Bir sonraki slaytta bu formülü göreceğiz.

2B Dizi Adres Hesaplama Formülü

- Satır-öncelikli (row-major) yerleşimde formül, 1B formülün doğal bir uzantısıdır.

```
adres(i,j) = taban_adres + (i × sütun_sayısı + j) × eleman_boyutu
```

(i × sütun_sayısı + j) ifadesi, satır ve sütun indekslerini TEK bir doğrusal indekse çevirir — sonrasında tanıdık 1B formül uygulanır. Bu yüzden önce i. satırın başına (i×sütun_sayısı), sonra o satır içinde j kadar ilerlenir.

2B Dizi Adres Hesaplama — Örnek

int m[3][4]; taban_adres=2000 sizeof(int)=4

m[0][0]	m[0][1]	m[0][2]	m[0][3]
m[1][0]	m[1][1]	**m[1][2]**	m[1][3]
m[2][0]	m[2][1]	m[2][2]	m[2][3]

```
adres(1,2) = 2000 + (1×4 + 2) × 4
```

```
adres(1,2) = 2000 + 6 × 4
```

```
adres(1,2) = 2000 + 24
```

adres(1,2) = 2024

2B Dizide C Bildirimi ve Erişim

```
int m[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

// Tüm elemanları gezmek için iç içe döngü:
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%d ", m[i][j]);
    }
}
```

Karmaşıklık: satır × sütun toplam eleman için O(satır × sütun) — iki iç içe döngü nedeniyle.

Seyrek Matris (Sparse Matrix) Kavramı

- Elemanının çoğunluğu SİFIR olan büyük matrislere seyrek matris denir.

0	0	5	0	0
0	0	0	0	0
3	0	0	0	8
0	0	0	0	0
0	0	0	1	0

Problem: 5×5=25 hücrenin sadece 4'ü dolu (%16). Büyük matrislerde (ör. 10000×10000) bu israf devasa boyutlara ulaşır.

Seyrek Matris Temsili — Üç-Sütunlu Yöntem

- Sadece SIFIR OLMAYAN değerler; (satır, sütun, değer) üçlüsü olarak saklanır.

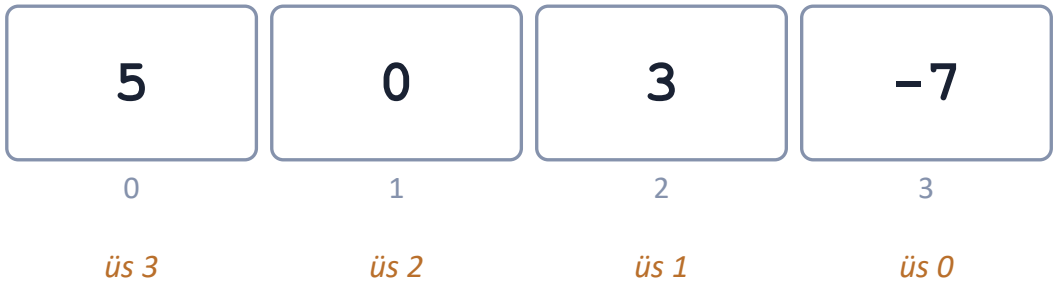
Satır	Sütun	Değer
0	2	5
2	0	3
2	4	8
4	3	1

Kazanç: 25 hücre yerine sadece 4 satır (üçlü başına 3 değer = 12 sayı) saklanır — büyük matrislerde devasa bellek tasarrufu sağlar.

Polinom Gösterimi (Dizi ile)

- Bir polinomun katsayıları, üs derecesine göre bir dizide saklanabilir.

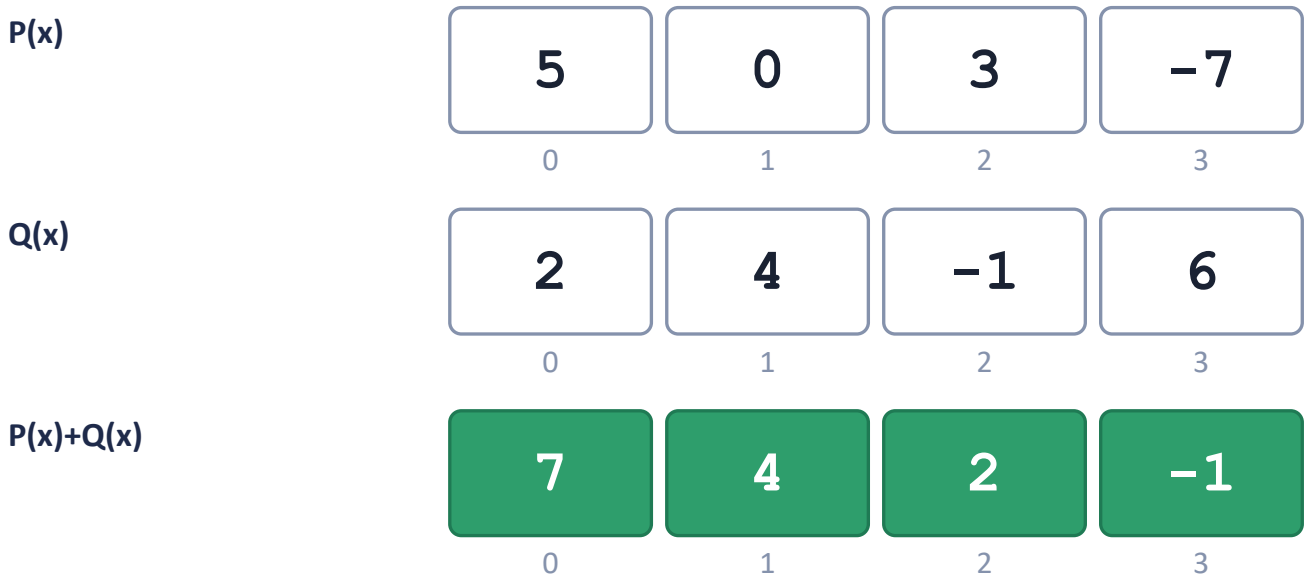
$$P(x) = 5x^3 + 0x^2 + 3x - 7$$



katsayi[i], x'in (derece−i) kuvvetinin katsayısını tutar — dizinin indeksi ile üs derecesi arasında düz bir ilişki vardır.

Polinom Toplama — Kısa Örnek

- Aynı dereceye sahip katsayılar teker teker toplanır.



P(x) = $5x^3+3x-7$, Q(x) = $2x^3+4x^2-x+6$ → Toplam = $7x^3+4x^2+2x-1$. Karmaşıklık: O(n) — her katsayı tam olarak bir kez toplanır.

Union Kavramı

```
union Deger {
    int i;
    float f;
    char c;
};

union Deger d;
d.i = 42;    // aynı 4 bayt şimdi
             // int olarak yorumlanıyor
```

Struct vs Union — Bellek Kullanımı

struct: her alan KENDİ ayrı bellek alanını alır (toplanır)

union: TÜM alanlar AYNI bellek alanını PAYLAŞIR (en büyük alan kadar yer kaplar)



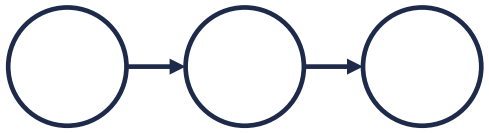
Kullanım alanı: aynı bellek bölgesinin farklı zamanlarda farklı türlerde yorumlanması gereken (bellek tasarrufu öncelikli) düşük seviye uygulamalar.

Haftalık Özet

- ✓ Dizinin bellekte bitişik yerleşimini ve adres hesaplama formülünü öğrendik.
- ✓ C'de dizi, işaretçi ve struct tanımlayıp kullandık.
- ✓ İşaretçi aritmetiği ile dizi erişimi arasındaki bağlantıyı gördük.
- ✓ Öz-referanslı yapılarla 3. haftaya (Bağlı Listeler) köprü kurduk.
- ✓ Ortaya ekleme ve silmeyi 4+3 adımda izleyip O(n) maliyetini kanıtladık.
- ✓ Çok boyutlu dizi, seyrek matris, polinom ve union'u temel düzeyde tanıdık.

Gelecek Hafta Önizleme

3. Hafta: Bağlı Listeler



- Tekli, çiftli ve dairesel bağlı listeler
- Dizi ile karşılaştırma: artık başa ekleme O(1)!
- **Bu hafta gördüğümüz öz-referanslı struct doğrudan kullanılacak**
- Düğüm (node) ekleme/silme işlemleri adım adım

Hazırlık: Bu haftaki 21. slaytta gördüğümüz "struct Dugum" yapısını tekrar edin — 3. hafta doğrudan buradan devam edecek.

Kavrama Soruları / Mini Quiz

1) taban_adres=500, sizeof(int)=4 olan bir dizide dizi[7]'nin adresi nedir?

2) Dizinin ortasına eleman eklerken kaydırma neden SONDAN BAŞA yapılır? Ters sırada ne olurdu?

3) int *p; ile int p; arasındaki fark nedir? p bir değer mi, bir adres mi tutar?

4) Bir struct kendi türünden bir işaretçi alanı neden içerebilir, ama kendi türünden doğrudan bir DEĞER alanı içeremez?

Haftalık Ödev

- 1) taban_adres=800, int dizisi için dizi[3] ve dizi[9]'un adreslerini hesaplayıp gösteriniz.
- 2) C dilinde, kapasitesi 10 olan bir tam sayı dizisi üzerinde ekleSonuna, ekleOrtaya(konum), sil(konum) ve gezin fonksiyonlarını yazınız.
- 3) [12, 25, 7, 31, 9] dizisinde konum=2'ye 99 eklemeyi, bu haftaki slayt formatında (her adım ayrı gösterilerek) elle çözünüz.
- 4) struct Ogrenci { int numara; char isim[20]; float ort; }; tanımlayıp 5 kişilik bir dizi oluşturan ve en yüksek ortalamaya sahip öğrenciyi bulan bir C programı yazınız.

Teslim: Bir sonraki ders öncesi, C kaynak dosyaları + elle çözülmüş adımları gösteren bir PDF olarak.

Kaynakça

Kaynak	İlgili Bölüm(ler)	Sayfa Aralığı
Horowitz, Sahni, Anderson-Freed — Fundamentals of Data Structures in C	Chapter 2: Arrays and Structures	s. 49–100
Thareja — Data Structures Using C	Chapter 3: Arrays; Chapter 5: Structures and Unions (ilgili kısımlar)	s. 66–114; 138–161

Not:

Adres hesaplama örnekleri, ekleme/silme animasyonları ve pointer/struct diyagramları özgün olarak hazırlanmıştır; kaynaklardaki ilgili bölümler kavramsal çerçeve için referans alınmıştır.